DOCUMENTATION

Dorker

Générateur de recherches avancées (dorks) pour Google, DuckDuckGo, Yandex, Twitter / X et Reddit.

Script Python, bibliothèque standard uniquement.

Octobre 2026

Sommaire

1 Présentation

1.1 Moteurs pris en charge

1.2 Prérequis

1.3 Lancer le script

1.4 Exemple rapide

2 Utilisation

2.1 Déroulé

2.2 Questions communes

2.3 Questions selon le moteur

2.4 Format des dates

2.5 Ouverture dans le navigateur

3 Opérateurs

3.1 Tableau des opérateurs

3.2 Même filtre, mots différents

3.3 URL de base

4 Fonctionnement du code

4.1 Imports

4.2 La fonction `demander_date`

4.3 La boucle principale

4.4 Les questions et les opérateurs

4.5 La construction des URL

1 Présentation

Dorker est un petit script Python qui transforme des réponses à des questions en **recherche avancée** (un "dork") et ouvre le résultat dans le navigateur.

Au lieu de taper à la main `"Romain Burel" -linkedin site:exemple.com filetype:pdf after:2026-01-01`, on répond à quelques questions et le script construit l'URL.

1.1 Moteurs pris en charge

- Google
- DuckDuckGo
- Yandex
- Twitter / X
- Reddit
- `tous` : lance la même recherche sur les cinq d'un coup

1.2 Prérequis

- Python 3
- Aucune bibliothèque à installer : le script n'utilise que la bibliothèque standard (`urllib.parse`, `webbrowser`).

1.3 Lancer le script

```
python dorker.py
```

Le script pose ses questions dans le terminal. Appuyer sur **Entrée** sans rien taper passe une question. Taper **q** au choix du moteur quitte le programme.

Chaque recherche s'ouvre automatiquement dans un nouvel onglet du navigateur, et son URL est aussi affichée dans le terminal.

1.4 Exemple rapide

```
Sur quel moteur? google
Quel mot veut tu chercher? Romain Burel
Une variante? R. Burel
Cette liste de caractère précisément? O/N o
Des mots a exclure? linkedin
Un site en particulier? exemple.com
Un type de fichier? pdf
Un mot dans le titre de la page?
Un mot dans l'adresse de la page?
A partir de quelle date? 06/10/2026
Jusqu'a quelle date?
```

Résultat (ouvert dans le navigateur) :

```
https://www.google.com/search?q=%22Romain+Burel%22+OR+%22R.+Burel%22+-
linkedin+site%3Aexemple.com+filetype%3Apdf+after%3A2026-10-06
```

2 Utilisation

2.1 Déroulé

1. Choisir un moteur : `google`, `duckduckgo`, `yandex`, `twitter`, `reddit`, `tous`, ou `q` pour quitter. Une réponse invalide est redemandée. Les majuscules sont acceptées.
2. Répondre aux questions communes.
3. Répondre aux questions propres au moteur choisi.
4. Le script affiche l'URL de chaque moteur, l'ouvre dans un nouvel onglet, puis revient au choix du moteur.

2.2 Questions communes

Posées pour tous les moteurs.

Question	Exemple de réponse	Effet dans la recherche
Quel mot veut tu chercher ?	`Romain Burel`	`Romain Burel`
Une variante ?	`R. Burel`	`... OR R. Burel`
Terme exact ? O/N	`o`	met des guillemets autour du mot et de la variante
Des mots à exclure ?	`linkedin wikipedia`	`-linkedin -wikipedia`

2.3 Questions selon le moteur

Seules les questions utiles au moteur choisi sont posées.

	Google	DuckDuckGo	Yandex	Twitter	Reddit
Site en particulier	x	x	x		
Type de fichier	x	x	x		
Mot dans le titre	x	x	x		
Mot dans l'adresse	x	x	x		
Compte (sans @ ni u/)				x	x
Subreddit (sans r/)					x
Date de début / de fin	x			x	

Avec `tous`, toutes les questions sont posées, et chaque moteur n'utilise que celles qu'il comprend.

La question "compte" sert à la fois pour Twitter et Reddit. En recherche de personne, c'est pratique : un même pseudo est souvent réutilisé d'un site à l'autre.

2.4 Format des dates

Les dates se tapent au format `DD/MM/YYYY`, par exemple `06/10/2026`.

Si la date est mal tapée, le script la redemande. Appuyer sur **Entrée** à ce moment-là abandonne la date.

2.5 Ouverture dans le navigateur

Chaque URL est ouverte dans un nouvel onglet avec `open_new_tab(url)`. Avec `tous`, cinq onglets s'ouvrent d'un coup.

Pour revenir à un simple affichage des URL, commenter cette ligne à la fin du fichier :

```
#open_new_tab(url)
```

3 Opérateurs

Un opérateur est un mot-clé que le moteur de recherche comprend comme un filtre. Le script les ajoute à la place de l'utilisateur.

3.1 Tableau des opérateurs

Opérateur	Rôle	Exemple	Moteurs
`"..."`	suite exacte de caractères	`"Romain Burel"`	tous
`OR`	l'un ou l'autre	`Burel OR Burrel`	Google, DuckDuckGo, Twitter, Reddit
`|`	l'un ou l'autre	`Burel | Burrel`	Yandex
`-mot`	exclure un mot	`-linkedin`	Google, DuckDuckGo, Yandex, Twitter
`site:`	limiter à un site	`site:exemple.com`	Google, DuckDuckGo, Yandex
`filetype:`	type de fichier	`filetype:pdf`	Google, DuckDuckGo
`mime:`	type de fichier	`mime:pdf`	Yandex
`intitle:`	mot dans le titre de la page	`intitle:cv`	Google, DuckDuckGo
`title:`	mot dans le titre de la page	`title:cv`	Yandex
`inurl:`	mot dans l'adresse de la page	`inurl:admin`	Google, DuckDuckGo, Yandex
`after:` / `before:`	après / avant une date	`after:2026-10-06`	Google
`from:`	tweets d'un compte	`from:rburel`	Twitter
`since:` / `until:`	depuis / jusqu'à une date	`since:2026-10-06`	Twitter
`author:`	posts d'un utilisateur	`author:rburel`	Reddit
`subreddit:`	limiter à un subreddit	`subreddit:osint`	Reddit

3.2 Même filtre, mots différents

Certains moteurs font la même chose avec un autre mot. Le script pose la question une seule fois, puis prépare chaque forme. Chaque moteur prend celle qu'il comprend.

Filtre	Google / DuckDuckGo	Yandex	Twitter	Reddit
L'un ou l'autre	`OR`	`|`	`OR`	`OR`
Type de fichier	`filetype:`	`mime:`		
Mot dans le titre	`intitle:`	`title:`		
Compte			`from:`	`author:`
Date de début	`after:` (Google)		`since:`	
Date de fin	`before:` (Google)		`until:`	

3.3 URL de base

Moteur	URL
Google	`https://www.google.com/search?q=`
DuckDuckGo	`https://duckduckgo.com/?q=`
Yandex	`https://yandex.com/search/?text=`
Twitter	`https://x.com/search?q=` + `&f=live` (tri par tweets récents)
Reddit	`https://www.reddit.com/search/?q=`

4 Fonctionnement du code

Le script se lit de haut en bas, en cinq parties.

4.1 Imports

```
from urllib.parse import quote_plus
from webbrowser import open_new_tab
```

`quote_plus` encode le texte pour l'URL : les espaces deviennent `+`, les guillemets `%22`, les `:` deviennent `%3A`.

4.2 La fonction `demander_date`

```
def demander_date(question):
    date = input(question)
    while date and (len(date.split("/")) != 3 or not date.replace("/", "").isdigit()):
        date = input("Invalide. (FORMAT: DD/MM/YYYY, Entree pour passer)")

    if date:
        d = date.split("/")
        date = d[2] + "-" + d[1] + "-" + d[0]

    return date
```

Renvoie la date au format `YYYY-MM-DD`, ou `""` si la question est passée.

La date est redemandée tant qu'elle ne fait pas 3 morceaux séparés par des `/`, ou qu'elle contient autre chose que des chiffres. Une réponse vide arrête la boucle.

4.3 La boucle principale

```
while True:
    moteur = input("Sur quel moteur? ...")
    ...
    if moteur == "q":
        break
```

Le choix du moteur est ensuite rangé dans une liste :

```
if moteur == "tous":
    choisis = moteurs
else:
    choisis = [moteur]
```

- `tous` donne `["google", "duckduckgo", "yandex", "twitter", "reddit"]`
- `google` donne `["google"]`

Comme `choisis` est toujours une liste, la partie 5 traite un ou cinq moteurs avec le même code.

4.4 Les questions et les opérateurs

Les variables optionnelles commencent vides, et chaque question n'est posée que pour les moteurs qui s'en servent :

```
if moteur in ["google", "duckduckgo", "yandex", "tous"]:
    site = input("Un site en particulier?")
```

Chaque réponse non vide reçoit ensuite son opérateur :

```
if site:
    site = "site:" + site
```

Une chaîne vide vaut `False` dans un `if` : une question passée n'ajoute rien à la recherche.

Quand deux moteurs utilisent des mots différents pour le même filtre, la réponse est préparée dans deux variables :

```
mime = ""
if filetype:
    mime = "mime:" + filetype          # pour Yandex
    filetype = "filetype:" + filetype  # pour Google et DuckDuckGo
```

Même principe pour `title:` / `intitle:`, `author:` / `from:`, et pour les dates.

4.5 La construction des URL

```
for m in choisis:
    if m == "google":
        url = "https://www.google.com/search?q=" + quote_plus(mots + " " + site + ...)
    ...
    print(url)
    open_new_tab(url)
```

Pour chaque moteur, les morceaux qu'il comprend sont assemblés, encodés avec `quote_plus` et collés derrière son URL.

Pour Yandex, `mots.replace(" OR ", " | ")` remplace `OR` par `|`.